

Diepe code-uitleg

De grafieken, regel voor regel

aquarium.js · languages.js · net.js · radar.js

Elke code-snipppet uitgelegd zodat je het zelf kunt aanpassen

Vervolg op "De Visdeurbel-datavisualisatie — stap voor stap".

Bestanden in `src/scripts/mitchell/charts/`

Zo lees je dit document

Per grafiek lopen we de code **van boven naar beneden** door in kleine blokjes. Onder elk blokje staat per regel wat hij doet — en waar nuttig een **Aanpassen-tip**.

- **Code-blokken** zijn de échte regels uit het bestand (donkere achtergrond).
- De **nummers** links in een blok verwijzen naar het regelnummer in het bronbestand.
- **Aanpassen** = een concrete knop waar je aan kunt draaien (kleur, snelheid, aantallen...).

Terugkerende begrippen — `const / let` = een waarde een naam geven (`const` = vast, `let` = mag wijzigen). `=>` = een "pijl-functie" (kort recept). `d3` = de tekenbibliotheek. `state` = de gedeelde data. `$('#id')` = zoek het HTML-element met dat id. Een volledige begrippenlijst staat achterin.

1 · aquarium.js CANVAS

src/scripts/mitchell/charts/aquarium.js

Tientallen visjes zwemmen rond op één `<canvas>`. Aantal per soort = hoe vaak gezien; grootte = gewicht. Klik = schrikken + rimpel, hover = naam & lengte.

1.1 — Imports (regel 1–5)

```
1 import { $, formatNumber, reduceMotion } from '../utils.js';
2 import { fishImagePath, hexToRgb01 } from '../fishImage.js';
3 import { showTooltip, hideTooltip } from '../tooltip.js';
4 import { state, lifecycle, raf } from '../state.js';
```

`$` = element opzoeken; `formatNumber` = getal met punten; `reduceMotion` = "animaties uit?".

`fishImagePath` = pad naar de vis-PNG; `hexToRgb01` = hex-kleur → 3 getallen 0–1 (voor het inkleuren).

`showTooltip/hideTooltip` = de gedeelde tooltip tonen/verbergen.

`state` = gedeelde data, `lifecycle` = opruimlijst, `raf` = animatieframe dat netjes te stoppen is.

1.2 — Start & canvas opzetten (regel 13–20)

```
13 export function initAquarium() {
14   const { TOTAL, visData } = state;
15   const { cleanups } = lifecycle;
16   const stage = $('#aquariumStage');
17   if (!stage) return;
18   stage.querySelectorAll('canvas, .aquarium-counter').forEach(n => n.remove());
19   const canvas = document.createElement('canvas');
20   stage.appendChild(canvas);
```

14 Haal `TOTAL` (totaal waarnemingen) en `visData` (de vissoorten) uit de gedeelde data.

15 Pak de opruimlijst, zodat we later onze timers/animaties kunnen afmelden.

16–17 Zoek het podium-`div`; bestaat het niet, stop dan meteen (voorkomt fouten).

18 Ruim een eventueel oud canvas/teller op (bij hertekenen na periodewissel).

19–20 Maak een nieuw `<canvas>` en hang het in het podium.

1.3 — Teller, tekencontext & scherpte (regel 21–35)

```
21 const counter = document.createElement('div');
22 counter.className = 'aquarium-counter';
23 counter.setAttribute('aria-live', 'polite');
24 stage.appendChild(counter);
25 const ctx = canvas.getContext('2d');
26 const dpr = Math.min(window.devicePixelRatio || 1, 2);
27 let W = 0, H = 0;
28 function resize() {
29   const r = stage.getBoundingClientRect();
30   W = r.width; H = r.height;
31   canvas.width = W * dpr; canvas.height = H * dpr;
32   ctx.setTransform(dpr, 0, 0, dpr, 0, 0);
33 }
34 resize();
35 const ro = new ResizeObserver(resize); ro.observe(stage);
```

21–24 Maak het teller-vakje. `aria-live="polite"` laat voorleessoftware de oplopende teller rustig melden.

25 `ctx` is het "penseel": alle tekenopdrachten lopen hierover.

26 `dpr` = schermresolutie (retina = 2). Met `Math.min(..., 2)` kappen we op 2 af — hoger kost veel rekenkracht zonder zichtbaar verschil.

27 `W/H` = breedte/hoogte in CSS-pixels; nog 0, worden in `resize()` gevuld.

28–33 `resize()`: meet het podium, zet het canvas op `W·dpr × H·dpr` echte pixels en schaal het penseel met `setTransform` zodat je in gewone pixels kunt tekenen (scherp, niet wazig).

34–35 Eén keer meten, en met een `ResizeObserver` opnieuw meten als het venster verandert.

Aanpassen. Wil je scherper op zeer hoge schermen? Verhoog de `2` in regel 26 (kost performance). Een vaste canvas-grootte? Vervang `resize()` door vaste `W/H`.

1.4 — Totaal & samenvatting (regel 37–46)

```
37 const total = TOTAL || visData.reduce((s, v) => s + v.count, 0) || 1;
38 const totalEl = $('#aquariumTotal'); if (totalEl) totalEl.textContent = formatNumber(total);
41 const sortedSpecies = visData.slice().sort((a, b) => (b.count || 0) - (a.count || 0));
42 const top3 = sortedSpecies.slice(0, 3).map(s => s. naam).join(', ');
43–46 // zet samenvatting in #aquariumSummary
```

37 `total` = het totaal. Eerst `TOTAL`; is dat 0, tel dan alle `count`'s op (`reduce`); en `|| 1` voorkomt deling-door-nul later.

38 Toon het totaal in de kop (`#aquariumTotal`), netjes geformatteerd.

41 `slice()` maakt een **kopie** (anders sorteert je de gedeelde `visData` !), daarna sorteren op aantal, hoog→laag.

42 Pak de top 3 namen en plak ze met komma's aan elkaar voor de zin.

1.5 — Vis-plaatjes inkleuren: de sprite-cache (regel 48–70)

```
49 const sprites = {}, imgCache = {};
51 function loadImage(url) {
52   if (imgCache[url]) return imgCache[url]; // al geladen? hergebruik
53   const img = new Image(); img.src = url; imgCache[url] = img; return img;
54 }
58 function makeSprite(naam, color) {
59   const key = naam + color;
60   if (sprites[key]) return sprites[key]; // al ingekleurd? hergebruik
61   const img = loadImage(fishImagePath(naam));
62   if (!img.complete || !img.naturalWidth) return null; // nog niet binnen
66–70 // teken de PNG op een mini-canvas 'c' van 64px breed
```

49 Twee "kaartenbakken": `sprites` (ingekleurde resultaten) en `imgCache` (rauwe afbeeldingen). Cachen = werk maar één keer doen.

51–54 **loadImage**: is de afbeelding al geladen, geef 'm terug; anders start het laden en onthoud 'm.

59–60 De sleutel is `naam+color`; bestaat die al, klaar. Zo krijgt elke soort+kleur precies één ingekleurde versie.

62 Is de PNG nog niet binnen (`complete` / `naturalWidth`), geef `null` terug — de tekenlus probeert het dan volgende frame opnieuw.

66–70 Teken de foto op een onzichtbaar mini-canvas `c` (basisbreedte 64px, hoogte volgens beeldverhouding).

1.6 — Pixel-voor-pixel inkleuren (regel 76–93)

```
76 const L = 0.4, lift = 1.12;
77 const [r0, g0, b0] = hexToRgb01(color);
78–80 const tr = r0 + (1 - r0) * L; // kleur richting wit mengen (lichter)
82 const data = offCtx.getImageData(0, 0, pixW, pixH);
83 const arr = data.data; // [r,g,b,a, r,g,b,a, ...]
```

```

84   for (let i = 0; i < arr.length; i += 4) {
85       if (arr[i + 3] === 0) continue; // transparant → overslaan
86       const y = 0.299*arr[i] + 0.587*arr[i+1] + 0.114*arr[i+2]; // grijswaarde
87-89   arr[i] = Math.min(255, y*tr*lift); // grijs × doelkleur
91   offCtx.putImageData(data, 0, 0); sprites[key] = c; return c;

```

76 `L` = hoeveel de kleur richting wit gemengd wordt (lichter); `lift` = extra helderheid. Hogere waarden = lichtere visjes.

77 Zet de hex-kleur om naar drie getallen 0–1 (rood, groen, blauw).

78–80 Meng elke kleurcomponent richting wit: `kleur + (1-kleur)·L`.

82–83 Lees alle pixels uit. `arr` is één lange rij getallen: per pixel 4 waarden (rood, groen, blauw, doorzichtigheid).

84 Loop met stappen van 4 (één pixel per keer).

85 Volledig doorzichtige pixel? Overslaan (achtergrond blijft leeg).

86 Bereken de **grijswaarde** `y` met de standaard ogen-weging (groen telt het zwaarst). Zo behouden we licht/donker = textuur.

87–89 Vermenigvuldig die grijswaarde met de doelkleur → de vis krijgt zijn kleur maar houdt zijn schaduwen. `Math.min(255,...)` voorkomt te-felle pixels.

91 Zet de bewerkte pixels terug, bewaar in de cache en geef de ingekleurde mini-canvas terug.

Aanpassen. Donkerder/feller visjes: verlaag `L` of `lift` (regel 76). Een heel andere kleur per soort regel je in `constants.js` (`color`), niet hier.

1.7 — Grootte op schaal & de steekproef van ~80 (regel 100–116)

```

100  const sizeForWeight = (kg) => Math.cbrt(kg) * 0.85;
103  const sample = [];
104  visData.forEach(v => {
105      const n = Math.max(1, Math.round((v.count / total) * 80));
106      for (let i = 0; i < n; i++) sample.push({
107          x: Math.random()*100, y: Math.random()*100,
108          vx: (Math.random()-0.5)*0.4, vy: (Math.random()-0.5)*0.2,
109          shape: v.shape, color: v.color, naam: v.naam,
110          size: sizeForWeight(v.weight),
111          lengthCm: Math.round((v.lengte || 30) * (0.82 + Math.random()*0.36)),
112          wiggle: Math.random()*Math.PI*2, visible: true,
113      });
114  });
115  });
116  });

```

100 **sizeForWeight**: grootte = $\sqrt[3]{\text{gewicht}} \times 0.85$. Derdemachtswortel omdat massa met lengte³ groeit → juiste verhouding tussen soorten. De `0.85` is de algehele schaal.

104–105 Voor elke soort: aantal visjes = aandeel in het totaal × 80, minimaal 1 (anders verdwijnt een zeldzame soort).

107 Startpositie willekeurig in 0–100 (we werken in **procenten** van het kijkglas, niet in pixels — handig bij resize).

108 Startsnellheid `vx/vy` licht willekeurig (de `-0.5` maakt 'm even vaak negatief als positief = beide richtingen).

109–110 Bewaar vorm, kleur, naam en de berekende grootte.

113 Lengte per individu: typische soort-lengte (`v.lengte`) ± ~18% toeval, voor de hover-tekst.

114 `wiggle` = een willekeurige start-fase voor de zwem-golf; `visible` = aan/uit via de filters.

Aanpassen. Meer/minder visjes: verander de `80` (regel 105). Algehele grootte: de `0.85` (regel 100). Sneller startend: de `0.4/0.2` (regel 108).

1.8 — Filter-chips per soort (regel 118–134)

```
122 visData.forEach(v => {
123-126 // maak een knop met de soort-naam en kleur
127   chip.addEventListener('click', () => {
128     const muted = chip.classList.toggle('muted');
129     chip.setAttribute('aria-pressed', String(!muted));
130     sample.forEach(f => { if (f.naam === v.naam) f.visible = !muted; });
131   });
132 });
```

122–126 Maak per soort een knopje (chip) met de naam en de soort-kleur (via een CSS-variabele `--chip`).

128 `classList.toggle('muted')` zet de gedempte stijl aan/uit én geeft terug of hij nu aan staat.

129 Houd het toegankelijkheids-attribuut `aria-pressed` in sync.

130 Zet alle visjes van die soort op zichtbaar/onzichtbaar — de tekenlus slaat onzichtbare over.

1.9 — Klik = schrikken + rimpel (regel 137–151)

```
138   const rect = canvas.getBoundingClientRect();
139-140   const px = (e.clientX-rect.left)/rect.width*100; // muis → procent-coördinaten
141   sample.forEach(f => {
142     const dx = f.x-px, dy = f.y-py, dist = Math.sqrt(dx*dx + dy*dy);
143     if (dist < 35) { const k = (35-dist)/35; f.vx += (dx/Math.max(0.1,dist))*k*2.5; f.vy += ...
*k*1.6; }
144   });
145-150 // maak een rimpel-element op de klikplek, verwijder na 950ms
```

138–140 Reken de muispositie om naar dezelfde procent-coördinaten als de visjes.

142 Bereken per vis de afstand tot de klik (`dist` via Pythagoras).

143 Binnen straal 35: geef een duwtje wég van de klik. `k` = sterkte (dichterbij = sterker). `dx/dist` is de richting; `Math.max(0.1,...)` voorkomt delen door 0.

145–150 Maak een `<span class="aquarium-rip">` op de klikplek (de CSS doet de rimpel-animatie) en ruim 'm na 950ms op.

Aanpassen. Groter schrikbereik: de `35` (regel 142–143). Heftiger wegschieten: de `2.5 / 1.6`. De rimpel-stijl zelf zit in CSS onder `.aquarium-rip`.

1.10 — Hover = naam & lengte (regel 153–162)

```
157   let closest = null, closestDist = 4;
158   sample.forEach(f => { if (!f.visible) return; const d = Math.hypot(f.x-px, f.y-py); if (d <
closestDist) { closestDist = d; closest = f; } });
159   if (closest) { ...; showTooltip(`<strong>${closest.naam}</strong>~${closest.lengthCm} cm...`,
e.clientX, e.clientY); }
160   else { ...; hideTooltip(); }
```

157 Zoek het dichtstbijzijnde visje, maar alleen als het binnen 4 (procent) ligt — anders geen tooltip.

158 Loop alle zichtbare visjes langs en onthoud de dichtstbijzijnde (`Math.hypot` = afstand).

159 Is er één dichtbij: cursor → pointer en toon de tooltip met **naam + ~lengte cm**.

160 Anders: cursor → kruisje en verberg de tooltip.

Aanpassen. Tooltiptekst zit in regel 159 — voeg bijv. `${closest.naam}` 's gewicht toe. Groter "aanwijs-bereik": verhoog de `4` (regel 157).

1.11 — De tekenlus `tick()` (regel 165–189)

```
166     ctx.clearRect(0, 0, W, H); // scherm leegvegen
167–169 // teken een zachte radiale gloed als achtergrond
170     sample.forEach(f => {
171         f.x += f.vx; f.y += f.vy + Math.sin(f.wiggle)*0.04; f.wiggle += 0.08;
172         f.vx += (50-f.x)*0.00006; f.vy += (50-f.y)*0.00006;
173         f.vx += (Math.random()-0.5)*0.01; ... // dwalen
174         f.vx *= 0.992; f.vy *= 0.992; // wrijving
175         if (f.x < -5) f.x = 105; ... // rand → andere kant
176         if (!f.visible) return;
177     });
178     const sprite = makeSprite(f.naam, f.color); if (!sprite) return;
179     const px = (f.x/100)*W, py = (f.y/100)*H; // procent → pixels
180     const angle = Math.atan2(f.vy, f.vx); const flip = Math.abs(angle) > Math.PI/2 ? -1 : 1;
181     ctx.save(); ctx.translate(px,py); ctx.rotate(...); ctx.scale(1, flip); ctx.drawImage(sprite, ...);
182     ctx.restore();
183     if (running) rafId = raf(tick); // volgende frame
```

166 Veeg het hele canvas leeg vóór een nieuw frame (anders smeren de visjes uit).

167–169 Teken een zachte turquoise gloed in het midden als achtergrond.

172 Beweeg: tel snelheid bij de positie op; `Math.sin(wiggle)` geeft een zachte verticale golf; `wiggle` tikt door.

173 Zachte trek naar het midden (50%): hoe verder weg, hoe sterker het tikje terug.

174 Een vleugje toeval = "dwalen", zodat het natuurlijk oogt.

175 Wrijving: snelheid × 0.992 per frame, anders blijven ze versnellen.

176–177 Zwemt een vis voorbij de rand (–5 of 105), zet 'm aan de overkant terug (eindeloos kijkglas).

178 Onzichtbaar (uitgefilterd)? Niet tekenen.

179–180 Pak de ingekleurde sprite; nog niet geladen → dit frame overslaan.

181 Reken procent-positie om naar pixels op het canvas.

182–183 `angle` = kijkrichting uit de snelheid; `flip` spiegelt de vis als hij naar links zwemt (anders zwemt hij achterstevoren).

185–186 `save/translate/rotate/scale/drawImage/restore` = teken de sprite op de juiste plek, gedraaid en evt. gespiegeld. `save/restore` isoleren deze transformatie.

188 Vraag het volgende frame aan — maar alleen als `running` nog waar is (zo stopt de lus netjes).

Aanpassen. Rustiger zwemmen: verhoog de wrijving richting 0.98 (regel 175) of verklein het dwalen (174). Sterkere samenscholing in het midden: verhoog 0.00006 (173).

1.12 — Teller-animatie & aan/uit met de scroll (regel 190–213)

```
191–194 // animateCounter: telt in 3800ms soepel op naar 'total' (ease-curve)
195     const observer = new IntersectionObserver([entry] => {
196         if (entry.isIntersecting) { if (!running && !reduceMotion()) { running = true; tick(); }
197         // teller één keer animeren; bij 'animaties uit' meteen eindstand
198     } else { running = false; cancelAnimationFrame(rafId); }
199     }, { threshold: 0.1 });
200     observer.observe(stage);
201     cleanups.push(() => { running = false; cancelAnimationFrame(rafId); ...; observer.disconnect();
202     ro.disconnect(); });
```

191–194 **animateCounter** laat het getal in 3,8s soepel oplopen met een ease-curve (eerst versnellen, dan afremmen).

201–203 De `IntersectionObserver` start de tekenlus zodra het aquarium in beeld is (en animaties aan staan).

204–208 De teller wordt maar één keer geanimeerd (`dataset.animated`); bij "animaties uit" springt hij meteen naar de eindstand.

209 Scrollt het weg: stop de lus en annuleer het frame (spaart de accu).

210 `threshold: 0.1` = "in beeld" zodra 10% zichtbaar is.

212 Opruimwerk: stop alles en koppel observers los. Belangrijk vóór een periodewissel.

2 · languages.js D3 · SVG

src/scripts/mitchell/charts/languages.js

Per taal een zwevend begroetingswoord; lettergrootte = aantal bezoekers. Een *D3-force-simulatie* duwt de woorden uit elkaar, daarna dobberen ze zacht.

2.1 — Data klaarzetten (regel 13–27)

```
14 const { languagesData } = state;
15 const stage = $('#langStage');
16 const list = (languagesData || []).slice(0, 24).map(d => {
17   const g = GREETINGS[d.code] || ['', d.code.toUpperCase()];
18   return { code: d.code, n: d.n, greeting: g[0], name: g[1] };
19 });
20
21 if (!list.length) { stage.innerHTML = '...Geen taaldata.'; return; }
22 const total = d3.sum(list, d => d.n);
23 const maxVisitors = d3.max(list, d => d.n);
24 const fontScale = d3.scaleSqrt().domain([0, maxVisitors]).range([15, 70]);
25 const palette = [COLORS.green, COLORS.greenMid, COLORS.teal, COLORS.pink, COLORS.goldDeep];
```

14–16 Pak de talen-data uit `state` en het podium.

17 Neem de **top 24** talen (`slice(0,24)`) en zet ze om naar handige objecten.

18 Zoek de begroeting + nette naam op in de `GREETINGS`-tabel; onbekend → leeg woord + landcode in hoofdletters.

19 Per taal: `code`, `n` (aantal bezoekers), `greeting` (het woord), `name` (de taalnaam).

21 Geen data? Toon een nette melding en stop.

22–23 `total` (som) en `maxVisitors` (grootste) — nodig voor percentages en de schaal.

25 **fontScale**: zet bezoekers om naar lettergrootte 15–70px. `scaleSqrt` = wortelschaal, zodat de **oppervlakte** eerlijk meeschaalt.

27 Vijf kleuren waar de woorden uit kiezen (bewust geen paars: dat valt weg op de paarse sectie).

Aanpassen. Meer talen tonen: de `24` (regel 17). Grotere woorden: de range `[15, 70]` (regel 25). Andere kleuren: de `palette` (regel 27).

2.2 — Afmetingen & startposities (regel 29–36)

```
29 const W = 900, H = 540, cx = W/2, cy = H/2;
30 list.forEach((d, i) => {
31   d.size = fontScale(d.n);
32   d.color = palette[i % palette.length];
33   d.rad = Math.max(d.greeting.length * d.size * 0.3, d.size * 0.62) + 6;
34   d.x = cx + (Math.random()-0.5)*260;
35   d.y = cy + (Math.random()-0.5)*160;
36 });
```

29 Vast tekenvlak 900×540 (de SVG schaalt later mee met het scherm); `cx / cy` = midden.

31 Bereken en bewaar de lettergrootte per woord.

32 Kies een kleur; `i % palette.length` loopt netjes rond door de 5 kleuren.

33 `d.rad` = geschatte "botsstraal" van het woord (breder bij langere/grotere woorden). De botsing-kracht gebruikt dit straks om overlap te voorkomen.

34–35 Geef een willekeurige startplek rond het midden — de simulatie schikt ze daarna netjes.

2.3 — De woorden tekenen (regel 38–46)

```
38 const svg = d3.select(stage).append('svg').attr('viewBox', `0 0 ${W} ${H}`);
39 const node = svg.selectAll('g.lang-word').data(list).join('g')...
40   .attr('role', 'img').attr('aria-label', d => `${d.name}: ... bezoekers`);
41-43 node.append('text')...attr('font-size', d => d.size)...text(d => d.greeting);
44-46 node.filter(d => d.size > 30).append('text')...text(d => d.name);
```

38 Maak de SVG met een `viewBox` — een intern coördinatenstelsel dat automatisch meeschaalt met de schermgrootte.

39 **Data-join**: koppel `list` aan `<g>`-groepen. Voor elke taal één groep. `.join('g')` maakt ze aan.

40 Toegankelijkheid: `role="img"` + `aria-label` zodat voorleessoftware "Japans: 1.234 bezoekers" zegt i.p.v. het rauwe teken.

41–43 Voeg het grote begroetingswoord toe; `font-size` en kleur komen per taal uit de data.

44–46 Alleen bij grotere woorden (`size > 30`) een klein taalnaam-label eronder — anders wordt het te druk.

2.4 — Tooltip (regel 48–51)

```
48 node.on('mouseenter mousemove', (e, d) => showTooltip(
49   `<strong>${d.name}</strong>${formatNumber(d.n)} bezoekers · ${(d.n/total*100).toFixed(1)}%`,
   e.clientX, e.clientY))
50   .on('mouseleave blur', () => hideTooltip())
51   .on('focus', (e, d) => { ... showTooltip(..., bb.left + bb.width/2, bb.top); });
```

48–49 Bij hover: tooltip met naam, aantal en **percentage** (`d.n/total*100`, op 1 decimaal).

50 Muis eraf of focus weg → tooltip verbergen.

51 Bij toetsenbord-focus de tooltip boven het woord plaatsen (`getBoundingClientRect` geeft de schermpositie).

2.5 — De force-simulatie (regel 56–61)

```
56 const simulation = d3.forceSimulation(list)
57   .force('x', d3.forceX(cx).strength(0.04))
58   .force('y', d3.forceY(cy).strength(0.06))
59   .force('charge', d3.forceManyBody().strength(-4))
60   .force('collide', d3.forceCollide(d => d.rad).strength(0.9))
61   .on('tick', () => node.attr('transform', d => `translate(${d.x},${d.y})`));
```

56 Start een **force-simulatie**: een nep-natuurkunde die elk woord een positie geeft.

57–58 `forceX/forceY` trekken zacht naar het midden (`strength` = hoe hard; klein = los).

59 `charge` met negatieve sterkte = woorden stoten elkaar licht af (lucht ertussen).

60 `collide` met straal `d.rad` = harde grens die overlap voorkomt.

61 Bij elke **tick** (rekenstap) verplaats je elke groep naar zijn nieuwe `x,y` via een `transform`.

Aanpassen. Strakker op een kluitje: verhoog `strength` bij `x/y` (57–58). Meer lucht tussen woorden: maak `charge` negatiever (59) of vergroot `d.rad` (regel 33).

2.6 — Stilstaan (animaties uit) óf zacht dobberen (regel 63–80)

```
63 if (reduceMotion()) {
64   simulation.stop(); for (let i=0;i<220;i++) simulation.tick(); // in één keer uitrekenen
65   node.attr('transform', d => `translate(${d.x},${d.y})`);
66 } else {
67   simulation.on('end', () => {
```

```

68     list.forEach(d => { d.bx = d.x; d.by = d.y; d.ph = Math.random()*Math.PI*2; });
70-74 // bob(): laat elk woord met een sinus rond zijn rustplek (bx,by) deinen
76     let bobId = raf(bob);
77     cleanups.push(() => cancelAnimationFrame(bobId));
78   });
79 }
80 cleanups.push(() => simulation.stop());

```

63–65 Animaties uit? Reken de simulatie in één keer 220 stappen door (geen beweging op het scherm) en zet de woorden meteen op hun eindplek.

67 Anders: wacht tot de simulatie tot rust komt (`'end'`).

68 Bewaar de rustplek (`bx,by`) en geef elk woord een willekeurige fase `ph` (zodat ze niet synchroon deinen).

70–74 bob(): verplaats elk woord met een trage sinus/cosinus rond zijn rustplek = zacht dobberen. Draait elke frame via `raf` .

76–77 Start het dobberen en meld het aan bij de opruimlijst.

80 Stop de simulatie altijd bij opruimen.

Aanpassen. Heftiger dobberen: vergroot de `5` 'en in `bob` (regel 72). Sneller: verhoog de `0.5 / 0.45` .

src/scripts/mitchell/charts/net.js

Alle soorten als bellen in een net. Belgrootte = **aantal**, **gewicht/vis** of **biomassa** (aantal × gewicht). Wisselen laat de bellen soepel "morphen".

3.1 — Opzet & het zakkende net (regel 13–36)

```
16–19 const host = $('#netStage'); const info = $('#netInfo'); ... const svg =
d3.select(host).append('svg')...;
22–23 const netGroup = svg.append('g'); const meshBottom = 380;
24–28 for (... i ≤ 16 ...) netGroup.append('path')...; // verticale touwen
29–32 for (... j ≤ 8 ...) netGroup.append('path')...; // horizontale touwen
35–36 netGroup.attr('transform', 'translate(0,-200)').transition()...attr('transform', 'translate(0,0)');
```

16–19 Pak het podium en de info-regel; maak de SVG.

22–23 Een groep voor het net; `meshBottom` = hoe ver het net naar beneden hangt.

24–28 Teken 17 verticale "touwen" als licht golvende paden (de `Math.sin(i)` geeft de slinger).

29–32 Teken 9 horizontale touwen, elk iets doorhangend (`Q` = een gebogen lijn).

35–36 Begin het net 200px hoger en laat het met een `transition` in beeld zakken (overgeslagen bij "animaties uit").

3.2 — De drie weergaven (regel 38–53)

```
40 let currentStat = 'biomass';
43 const statFn = { count: d => d.count, weight: d => d.weight, biomass: d => d.count * d.weight };
44–48 const statLabel = { ... }; // tekst per weergave
49–53 const explainer = { ... }; // uitleg-zin onderaan
```

40 Begin in de **biomassa**-weergave.

43 **statFn**: drie recepten voor de belgrootte. `biomass = count × weight`. Dit is het hart van de grafiek.

44–48 **statLabel**: de tekst in de info-regel per weergave (bijv. "... kg biomassa (aantal × gewicht)").

49–53 **explainer**: korte uitleg-zin die onder de grafiek verschijnt.

3.3 — Bellen rangschikken met circle packing (regel 58–63)

```
58 function renderPack(stat) {
59   const packData = visData.map(v => ({ ...v, value: statFn[stat](v) }));
61   const pack = d3.pack().size([W-40, H-100]).padding(8);
62   const root = d3.hierarchy({ children: packData }).sum(d => d.value);
63   const nodes = pack(root).leaves();
```

59 Geef elke soort een `value` volgens de gekozen weergave (de `...v` kopieert de bestaande velden).

61 **d3.pack** = de circle-packing-machine; `size` = beschikbaar vlak, `padding` = ruimte tussen bellen.

62 `d3.hierarchy(...).sum(...)` bouwt de boom en sommeert de `value`'s; de som bepaalt de cirkelgroottes.

63 `pack(root).leaves()` rekent de posities + stralen uit en geeft de losse bellen terug (de "bladeren").

3.4 — Data-join: enter / update / exit (regel 66–71, 89, 106–109)

```
66 const sel = bubbleGroup.selectAll('.net-bubble').data(nodes, d => d.data.naam);
67 sel.exit().transition()...attr('transform', `scale(0)`).remove(); // verdwenen → krimpen
69–71 const enter = sel.enter().append('g')...; // nieuw → aanmaken
```

```

89     const all = enter.merge(sel); // nieuw + bestaand samen
106-109 enter.transition()...scale(1); sel.transition()...scale(1); // naar eindplek animeren

```

66 Koppel de bellen aan de data, met de **naam als sleutel** (2e argument). Zo weet D3 welke bel bij welke soort hoort, ook na het wisselen van weergave.

67 **exit** = soorten die er niet meer zijn → laat ze naar `scale(0)` krimpen en verwijder.

69-71 **enter** = nieuwe bellen → maak een groep, klein (`scale(0)`) en 200px hoger (ze "vallen" straks in het net).

89 `enter.merge(sel)` = de nieuwe én bestaande bellen samen, zodat we ze in één keer kunnen bijwerken.

106-109 Animeer iedereen naar zijn eindplek en volle grootte (`scale(1)`). Hierdoor "morphen" de bellen bij een wissel.

Waarom dit slim is. Door de naam als sleutel te gebruiken, blijft elke bel "dezelfde" tussen weergaven en schuift hij netjes naar zijn nieuwe grootte/plek — in plaats van weg te knippen en opnieuw te verschijnen.

3.5 — De inhoud van een bel (regel 73-87)

```

73     enter.each(function (d) {
75-79     // maak een radiaal kleurverloop (gradiënt) in de soort-kleur
80-81 g.append('circle').attr('class','bub-main').attr('r', d.r).attr('fill', `url(#${gradId})`)...
82-83 g.append('circle').attr('class','bub-shine')... // glansplekje
84     g.append('g').attr('class','bub-fish')... // plek voor het vis-plaatje
85-86 g.append('text').attr('class','bub-label')... // het naam-label
87     });

```

73 `enter.each(...)` = doe dit één keer per nieuwe bel.

75-79 Maak een eigen radiaal kleurverloop (lichte kern → soort-kleur rand) en geef het een uniek `id`.

80-81 De hoofdcirkel, gevuld met dat kleurverloop. `d.r` = de door packing berekende straal.

82-83 Een klein lichter cirkeltje als "glans" linksboven.

84 Een lege groep waar straks het ingekleurde vis-plaatje in komt.

85-86 Het tekst-label met de soortnaam.

3.6 — Plaatje, info & soepel groeien (regel 90-105)

```

92-93 const fishSize = Math.min(d.r*1.4, 96); const tint = ensureTintFilter(svg, d.data.color);
94-96 g.select('.bub-fish').html(`<image href="#" filter="url(#${tint})"/>`);
98     const setInfo = () => { info.textContent = `${d.data.naam} - ${statLabel[stat](d.data)}.`; };
99-100 g.on('click', setInfo).on('mouseenter', setInfo).on('keydown', ...);
101-104 g.select('.bub-main').transition()...attr('r', d.r); ...label bijwerken...

```

92-93 Plaatjesgrootte ~1,4x de straal (max 96). `ensureTintFilter` levert het kleur-filter (uit `fishImage.js`).

94-96 Zet het vis-plaatje in de bel, ingekleurd via dat filter.

98 **setInfo**: schrijf de cijfers van deze soort in de info-regel.

99-100 Toon die info bij klik, hover én Enter/Spatie (toetsenbord).

101-104 Animeer de cirkel, de glans en het label naar hun nieuwe grootte. Het label vervaagt (`opacity 0`) als de bel klein is (`d.r < 28`) — anders wordt het onleesbaar.

3.7 — De schakelknoppen (regel 111-131)

```

111-112 renderPack(currentStat); info.textContent = explainer[currentStat]; // eerste render
115     $$('.net-toggle-btn').forEach(btn => {
119         btn.onclick = () => {
120             if (btn.dataset.stat === currentStat) return; // al actief? niets doen

```

```
121     currentStat = btn.dataset.stat;
122-126 // markeer de actieve knop
127-128 renderPack(currentStat); info.textContent = explainer[currentStat];
129     };
131     cleanups.push(() => { $$('.net-toggle-btn').forEach(b => { b.onclick = null; }); });
```

111-112 Teken de eerste keer en zet de uitleg-zin.

115 Loop over de drie knoppen (\$\$ = alle elementen).

119 `btn.onclick = ...` (i.p.v. `addEventListener`): zo overschrijf je bij hertekenen de oude handler i.p.v. er steeds één bij te stapelen.

120 Klik op de al-actieve knop? Niets doen.

121 Onthoud de nieuwe weergave en teken opnieuw (regel 127) → de bellen morphen.

131 Bij opruimen de klik-handlers losmaken (geen geheugenlek).

Aanpassen. Andere startweergave: de `'biomass'` (regel 40). Meer ruimte tussen bellen: de `padding(8)` (regel 61). Drempel waaronder het label verdwijnt: de `28` (regel 104).

4 · radar.js D3 · SVG · ANIMATIE

src/scripts/mitchell/charts/radar.js

Een sonar: elke soort is een "ping", afstand tot het midden = hoe vaak gezien. Een draaiende straal licht de pings op (naglooi). Een tijdslijder scrubt door dag/week/maand.

4.1 — Start & afmetingen (regel 16–20)

```
17 const { visData, weekHours, weekDayLabels, weekDays, currentPeriod } = state;
19 const W = 620, H = 620, cx = W/2, cy = H/2, R = 270;
20 const svg = d3.select('#radarStage').append('svg').attr('viewBox', `0 0 ${W} ${H}`);
```

17 Haal alles uit `state`: de soorten, de uren-per-dag, de daglabels/datums, en de actieve periode (week/maand/jaar) — die bepaalt straks de tijdvakken.

19 Vierkant vlak 620x620; `cx, cy` = midden; `R` = straal van de buitenste ring.

20 Maak de SVG met meeschalende `viewBox`.

4.2 — Decor: schijf, ringen & dradenkruis (regel 22–40)

```
23-24 svg.append('circle')...attr('r', R).attr('fill', 'rgba(1,70,60,0.03)'); // vulling
28 for (let i = 1; i <= 4; i++) {
29   const isEdge = i === 4;
30-34   svg.append('circle')...attr('r', R*i/4).attr('stroke', isEdge ? '0.40' : '0.22')...
36-40   // twee lijnen door het midden = het dradenkruis
```

23–24 Een heel lichte schijf-vulling zodat de radar zich aftekent tegen de achtergrond.

28–29 Teken 4 ringen op straal `R/4`, `R/2`, `3R/4`, `R`. `isEdge` = de buitenste (de 4e).

30–34 De buitenrand is donkerder/dikker en solide; de binnenringen lichter en gestippeld (`stroke-dasharray`).

36–40 Twee lijnen (horizontaal + verticaal) door het midden vormen het dradenkruis.

Aanpassen (contrast). Sterkere ringen: verhoog de `0.40 / 0.22` (regel 32) en de breedte `1.3` (regel 33).
Dradenkruis: de `0.16` (regel 39).

4.3 — De draaiende sweep (regel 42–50)

```
43-46 // een kleurverloop van transparant → groen voor de sweep-wig
47-49 const sweep = svg.append('path').attr('d', `M ${cx} ${cy} L ... A ...`)... // taartpunt
50   svg.append('circle')...attr('r', 5)...; // stip in het midden
```

43–46 Een lineair kleurverloop (gradiënt): doorzichtig aan de ene kant, groen aan de andere — geeft de sweep zijn "veeg".

47–49 De sweep zelf is een taartpunt-pad (`M` = ga naar, `L` = lijn, `A` = boog). We bewaren 'm in `sweep` om straks te draaien.

50 Een klein stipje markeert het exacte midden.

4.4 — Afstand-schaal & hulpfunctie (regel 57–63)

```
57 const pingSeed = mulberry32(42);
59 const maxCount = Math.max(...counts, 1);
60 const innerDist = 70, outerDist = R - 55;
61 const distanceScale = d3.scaleSqrt().domain([0, maxCount]).range([outerDist, innerDist]).clamp(true);
62 const distFor = (count, jitter) => count > 0 ? distanceScale(count) + jitter : outerDist + 10;
```

57 Een *seeded* toevalsgenerator met vaste startwaarde 42 → elke render dezelfde plaatsing.

59 Het drukste aantal (minimaal 1) is het ijkpunt van de schaal.

60 `innerDist` = dichtst bij het midden (70px), `outerDist` = bij de rand.

61 `distanceScale`: zet een aantal om naar een afstand. Let op de **omgekeerde** range `[outerDist, innerDist]`: 0 → buiten, `maxCount` → binnen. `clamp(true)` houdt het binnen de grenzen.

62 `distFor`: afstand voor een aantal, plus een vaste `jitter` (rommel) zodat pings niet op een perfecte cirkel liggen. Aantal 0 → net buiten de rand.

4.5 — Pings plaatsen zonder overlap (regel 65–83)

```
66 visData.forEach((v, si) => {
67   const jitter = (pingSeed()-0.5)*22;
68   const distance = distFor(v.count, jitter);
69-77   for (... t < 60 ...) { const angle = pingSeed()*Math.PI*2; ... const nearest = ...; if (nearest >=
minGap) break; ... }
78-82 pings.push({ ...v, ...best, si, jitter, total: v.count||0, current: ..., litAt: -1e9, hover: false,
selected: false });
```

67 Een vaste jitter per ping (zodat hij bij scrubben niet "rondspringt", alleen radiaal beweegt).

68 De startafstand op basis van het totaal-aantal.

69–77 **Rejection sampling**: probeer tot 60x een willekeurige hoek; bereken de afstand tot de dichtstbijzijnde al-geplaatste ping (`nearest`); is die groot genoeg (`minGap`), stop (`break`). Zo voorkom je dat soorten op elkaar landen. Lukt het niet, neem de beste pinging.

78–82 Bewaar de ping: alle soort-velden (`...v`), de gekozen hoek/positie (`...best`), de index `si`, de jitter, `total/current` (aantal nu), en de animatie-vlaggen `litAt` (wanneer voor het laatst opgelicht), `hover`, `selected`.

4.6 — Tijdvakken bouwen: dag-totalen & groeperen (regel 92–128)

```
93-99 // tel per dag alle 24 uren op → dayTotals[]
103-105 if (currentPeriod === 'week') groups = ...; // elke dag = 1 bucket
106-118 else if (currentPeriod === 'jaar') { ... per kalendermaand groeperen ... }
119-127 else { ... maand → weken van 7 dagen ... }
```

93–99 Maak `dayTotals`: per dag de som van de 24 uur-waarden uit `weekHours`. Dit is de echte "drukte-vorm".

103–105 **week**: elke dag wordt een eigen tijdvak (label = de datum).

106–118 **jaar**: groepeer de 365 dagen per kalendermaand (op basis van de datum); label = maandnaam.

119–127 **maand**: hak de ~31 dagen in blokken van 7 → "Week 1...5", met de datumreeks als sub-label.

4.7 — Verdeling per soort, met behoud van het totaal (regel 130–155)

```
131-133 const agg = groups.map(...sommeer dayTotals...); const weights = agg.map(v => v/aggSum);
137 visData.forEach((v, si) => {
138   const rnd = mulberry32(1000 + si*7);
139   const raw = weights.map(w => Math.max(0.0001, w*(0.5+rnd()))); // drukte-vorm + soort-
variatie
141-142 const exact = raw.map(x => (x/rawSum)*v.count); const rounded = exact.map(Math.round);
144-150 // afrondings-rest verdelen zodat de som exact v.count is
151   perSpecies.set(si, rounded);
155   return { labels, subs, agg, perSpecies, noun };
```

131–133 `agg` = totale drukte per tijdvak; `weights` = die drukte als fractie van het geheel (samen 1).

138 Per soort een eigen toevalsgenerator (zaad `1000 + si·7`) → stabiel maar per soort anders.

139 `raw` = de drukte-vorm (`weights`) met een soort-eigen variatie (`0.5 + rnd()`) eroverheen, zodat niet alle soorten identiek bewegen.

141–142 Schaal terug zodat de som = het soort-totaal, en rond af op hele vissen.

144–150 Afronden verandert de som; het verschil (`diff`) wordt over de grootste tijdvakken verdeeld zodat de som **exact** klopt.

151–155 Bewaar per soort de verdeling, en geef alles terug: labels, sub-labels, aggregaat, de verdeling en het zelfstandig naamwoord (week/maand/jaar).

Waarom afgeleid? Er is geen kant-en-klare data "soort × tijdvak". We gebruiken de échte drukte per dag als vorm en verdelen elk soort-totaal daarover, met een vleugje variatie. Realistisch én altijd kloppend met het totaal.

4.8 — Pings tekenen + info op hover (regel 161–208)

```
162–166 const g = pingsGroup.append('g')...attr('opacity', 0)...; // start onzichtbaar
167–168 g.append('circle')...r 18 (halo) ... g.append('circle')...r 8 (kern)
169–174 // ingekleurd vis-plaatje via ensureTintFilter
177–183 const labelTextEl = g.append('text')...text(`${p.naam} · ${formatNumber(p.count)}`);
185–192 const showDetail = () => { ... detailPanel.innerHTML = `...`; detailPanel.classList.add('visible');
};
196–201 const hideDetail = () => { p.hover=false; p.selected=false; ...classList.remove('visible'); };
202–205 g.on('mouseenter', ...showDetail).on('mouseleave', hideDetail).on('focus', showDetail).on('blur',
hideDetail);
```

162–166 Een groep per ping, **start onzichtbaar** (`opacity 0`) — de sweep onthult 'm straks.

167–168 Twee cirkels: een grote, vage halo (`r=18`) en een vollere kern (`r=8`) in de soort-kleur.

169–174 Het ingekleurde vis-plaatje, met het tint-filter uit `fishImage.js`.

177–183 Het label "naam · aantal". `labelRight` bepaalt of het rechts of links van de vis staat (zodat het niet buiten beeld valt). We bewaren het in `p.labelTextEl` om het bij scrubben te updaten.

185–192 **showDetail**: maak deze ping de geselecteerde (de rest niet), en vul het detailpaneel met naam, huidig aantal, piekmaand, diepte en gewicht.

196–201 **hideDetail**: alles terugzetten en het paneel verbergen.

202–205 Koppel: muis erop / focus → tonen; muis eraf / blur → verbergen. (Hover-info, geen klik.)

Aanpassen (info). Welke regels in het paneel staan, zit in `showDetail` (regel 190). Wil je info laten staan i.p.v. verdwijnen op `mouseleave`: haal het verbergen uit `hideDetail`.

4.9 — De tijdslijder opbouwen (regel 213–236)

```
214–222 const scrubber = document.createElement('div'); Object.assign(scrubber.style, { ... });
225–233 const range = document.createElement('input'); range.type='range'; range.max =
String(buckets.labels.length); range.value='0';
235 stageEl.insertAdjacentElement('afterend', scrubber); // ónder de radar
236 cleanups.push(() => scrubber.remove());
```

214–222 Maak een doosje met inline-stijl (gecentreerd, lichte achtergrond). Bewust géén absolute positie — zodat het de radar niet bedekt.

225–233 Een schuifbalk (`input type=range`) van 0 t/m het aantal tijdvakken; stap 1; start op 0 (= hele periode).

235 Plaats het **direct ná** de radar in de pagina (niet erin), dus het verschijnt eronder.

236 Bij opruimen weghalen (anders blijft een oude slider staan na periodewissel).

4.10 — Scrubben: pings herpositioneren (regel 239–266)

```
240–243 const isAll = pos === 0; sliderLabel.textContent = isAll ? `Hele ...` : `${buckets.labels[pos-1]}...`;
248     const viewCounts = pings.map(p => isAll ? p.total : buckets.perSpecies.get(p.si)?.[pos-1] ?? 0);
249–250 const viewMax = Math.max(...viewCounts, 1); const scale = d3.scaleSqrt().domain([0, viewMax])...;
252–257 pings.forEach((p, idx) => { const c = viewCounts[idx]; const dist = c>0 ? scale(c)+p.jitter : outerDist+10; p.x = cx + Math.cos(p.angle)*dist; ... });
258–262 p.elem.transition().attr('transform', ...); p.labelTextEl.text(...); if (p.selected) p.showDetail();
```

240–243 `pos 0` = hele periode; anders een tijdvak. Zet de labeltekst boven de slider.

248 Haal het aantal per soort op voor deze stand (totaal, óf uit de verdeling van dit tijdvak).

249–250 **Sleutel-stap:** maak een **nieuwe** schaal genormaliseerd op de drukste soort van *dit* tijdvak (`viewMax`). Daardoor staan de vissen altijd ten opzichte van elkáár, niet allemaal samengeperst aan de rand.

252–257 Bereken per ping de nieuwe afstand (met de vaste jitter) en daaruit `x,y` — de hoek `p.angle` blijft gelijk, dus hij beweegt alleen naar binnen/buiten.

258–262 Animeer soepel naar de nieuwe plek, werk het label-aantal bij, en ververs het detailpaneel als deze ping geselecteerd is.

Aanpassen. Wil je **absolute** afstanden (vergelijkbaar tussen tijdvakken) i.p.v. relatief? Gebruik in regel 250 weer `maxCount` i.p.v. `viewMax`. Tragere overgang: de `650` (regel 258).

4.11 — Sweep draaien & naglooi (regel 279–308)

```
279 const FL00R = 0.12, GLOW_MS = 1600;
282–284 function tick(now) { sweepAngle += reduceMotion() ? 0.03 : 0.01; sweep.attr('transform', `rotate(...)`);
286–287 const delta = (p.angle - sweepAngle + Math.PI*4) % (Math.PI*2); if (delta < 0.18) p.litAt = now;
290–297 if (reduceMotion()) op=1; else if (p.hover||p.selected) op=1; else if (p.litAt<0) op=0; else { ... glow ... }
298     p.elem.attr('opacity', op);
300     if (running) frameId = raf(tick);
303–307 // IntersectionObserver: start tick in beeld, stop buiten beeld
```

279 `FL00R` = laagste helderheid (nooit helemaal weg); `GLOW_MS` = hoe lang de naglooi duurt (1,6s).

282–284 Elke frame draait de sweep een stukje verder (`sweepAngle`) en past de rotatie toe.

286–287 `delta` = hoekverschil tussen ping en sweep. Veegt de sweep er net overheen (`< 0.18` rad), onthoud dán "nu opgelicht" (`p.litAt = now`).

290–297 Bepaal de helderheid: animaties uit → vol; hover/geselecteerd → vol; nog nooit gezien → onzichtbaar; anders een **uitdovende gloed** op basis van hoe lang geleden de sweep passeerde.

298 Pas die helderheid toe op de ping.

300 Vraag het volgende frame — alleen als `running`.

303–307 Start de lus als de radar in beeld is, stop 'm als hij weg scrolt (accu sparen).

Aanpassen (gevoel). Sneller draaien: de `0.01` (regel 283). Langer naglinsteren: `GLOW_MS` hoger. Pings nooit helemaal laten verdwijnen: `FL00R` hoger (bijv. 0.25).

Snelle begrippenlijst

Term	In gewone taal
canvas	Eén tekenvlak waarop je snel heel veel dingen tekent (aquarium). Je veegt het elk frame leeg en tekent opnieuw.
SVG	Losse, schaalbare vormen (cirkels, tekst, paden) — goed voor klikbare/animateerbare elementen (radar, net, talen).
ctx (context)	Het "penseel" van een canvas; alle tekenopdrachten lopen erover.
requestAnimationFrame / raf	Vraagt de browser vóór elke schermverversing (~60x/sec) een functie te draaien. De motor van elke animatielus.
IntersectionObserver	Meldt wanneer iets in/uit beeld scrolt. Gebruikt om animaties te starten/stoppen en grafieken pas te tekenen als je ze ziet.
scaleSqrt	Een D3-schaal die via de wortel rekent, zodat de <i>oppervlakte</i> eerlijk meeschaalt (twee keer zoveel = twee keer zo veel vlak).
force-simulatie	Nep-natuurkunde van D3: elementen trekken/duwen tot ze een nette verdeling vinden (talen).
circle packing (d3.pack)	Cirkels zo compact mogelijk in een vlak passen, op grootte (net).
data-join (enter/update/exit)	D3 koppelt data aan vormen: nieuw → enter, gewijzigd → update, weg → exit. Maakt soepele overgangen.
seeded random (mulberry32)	"Toeval" met een vaste startwaarde → elke keer dezelfde reeks. Zo staan pings/posities reproduceerbaar.
rejection sampling	Net zo lang een willekeurige plek proberen tot hij aan een eis voldoet (hier: niet te dicht op een andere ping).
jitter	Een beetje vaste rommel op een waarde, zodat dingen niet té perfect/regelmatig ogen.
transition	D3 animeert een eigenschap (positie, grootte, kleur) soepel van oud naar nieuw over een tijdsduur.
ease-curve	Versnellen/afremmen tijdens een animatie i.p.v. constante snelheid (voelt natuurlijker).
device pixel ratio (dpr)	Hoeveel echte pixels één CSS-pixel beslaat (retina = 2). Canvas wordt hierop afgesteld om scherp te blijven.
cleanups / lifecycle	Lijst met "opruim"-functies. Bij een periodewissel of paginaverlaten worden ze gedraaid zodat niets blijft hangen.

Tot slot. Alle vier de bestanden volgen hetzelfde stramien: **data uit state** → **vorm berekenen** → **tekenen** → **interactie koppelen** → **animeren met raf** → **opruimen via cleanups**. Herken je dat patroon, dan kun je elke grafiek lezen en aanpassen.